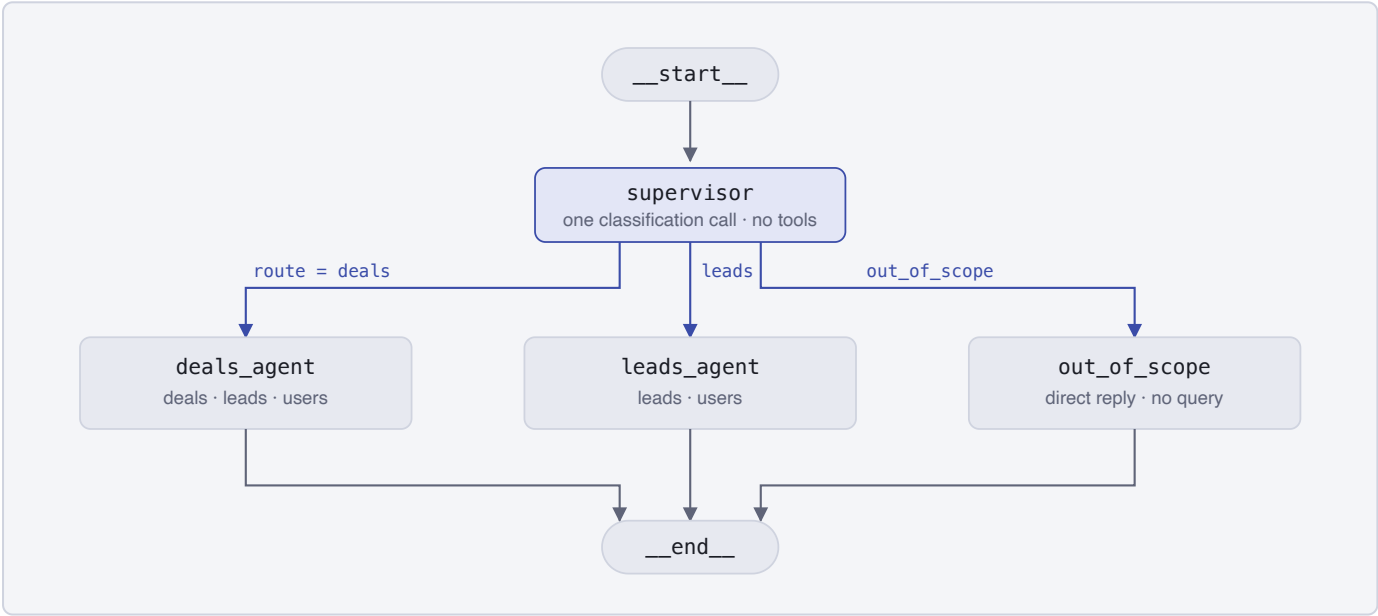


MarQ Agent

A read-only conversational layer over the MyTAI CRM. A supervisor routes each question to a domain specialist; the specialist reaches the database through a single guarded tool, and never writes SQL itself.

AGENTS 2 + 1 supervisor	MODEL CALLS / TURN 2–3 route · agent · SQL	TESTS 185 108 need no network	EVAL CASES 70 SQL 32 · graph 15 · route 23	WRITE PATHS 0 read-only by construction
--	---	--	---	--

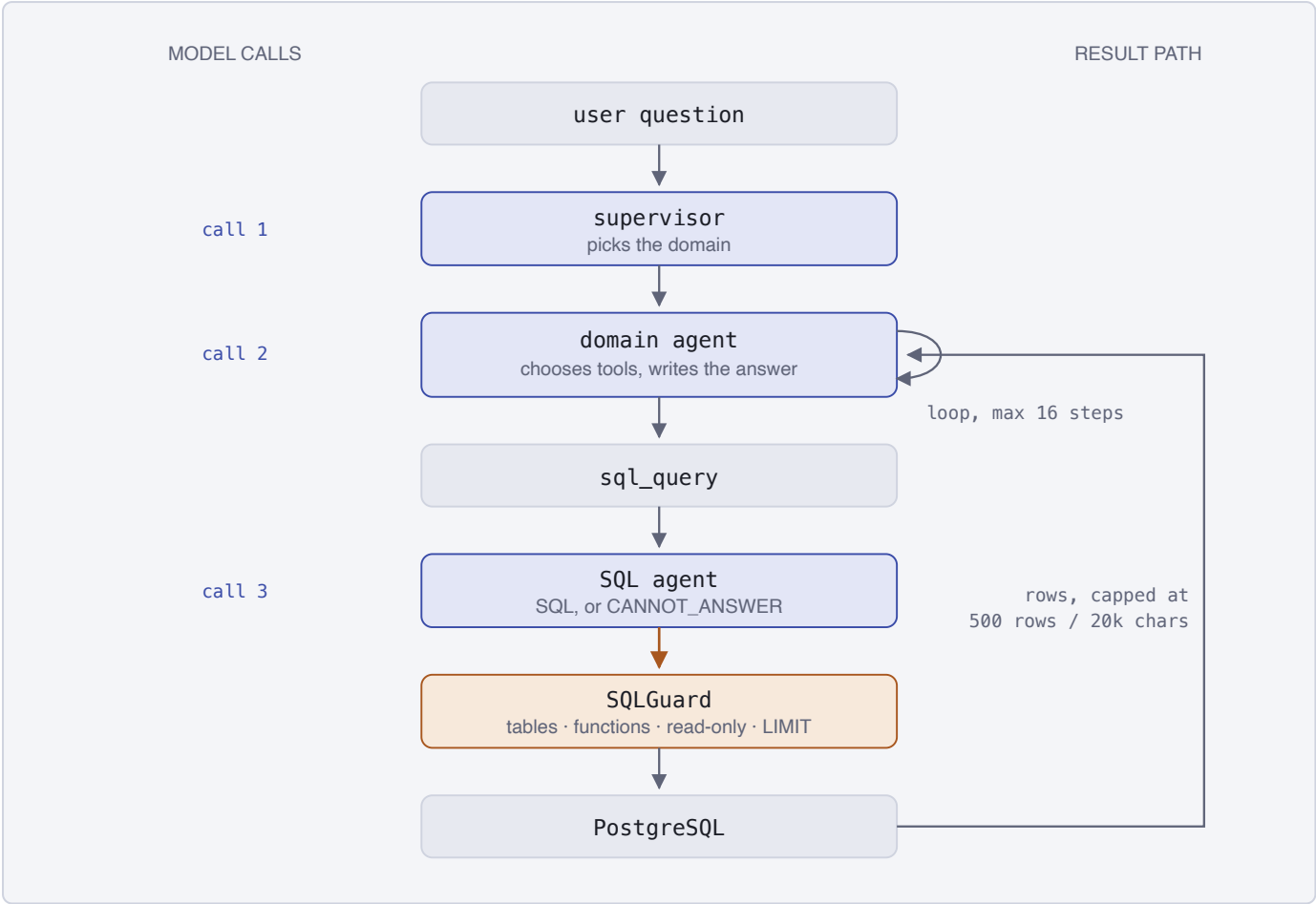
The graph



Routing is a single decision taken before any agent runs, written to AgentState.route so a wrong answer caused by a misroute is visible in the trace rather than inferred. Agents cannot hand off to each other — each one's guard permits only its own tables, so a misrouted question cannot be rescued mid-answer.

The route ladder is ordered. Any question involving deals goes to deals — even when the noun being counted is a lead, as in "how many leads became deals". Only the deals domain sees both tables. An unparseable classification falls back to deals for the same reason: a misroute there can still be answered.

What happens to one question

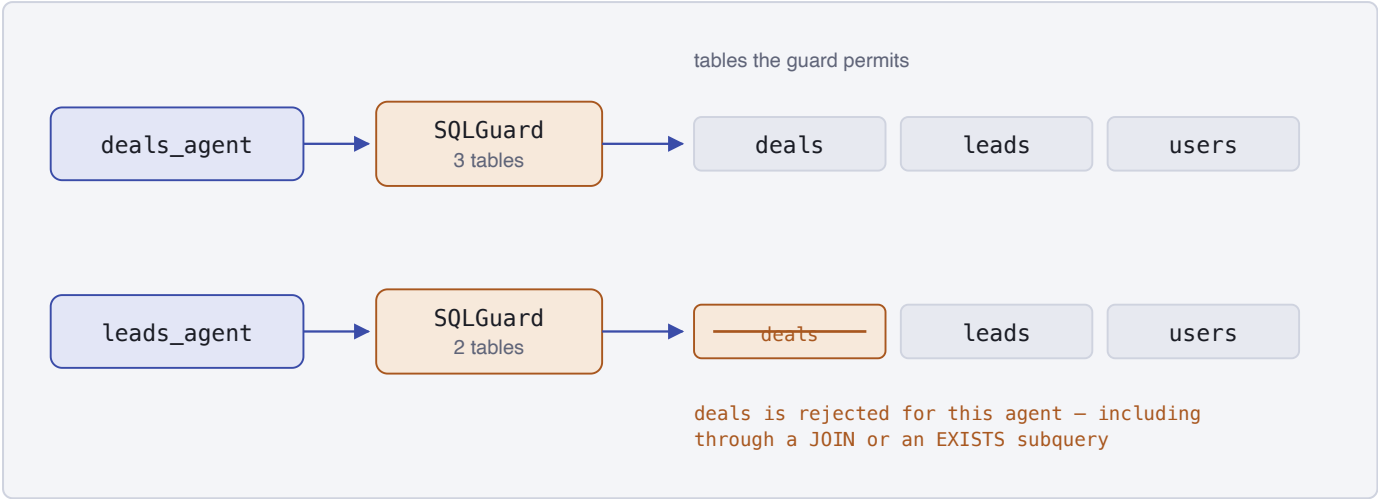


Two or three model calls per turn. The domain agent never sees SQL and the SQL agent never sees the conversation — which is why the agent must resolve pronouns and keep the user's own limits ("in the next 30 days") intact when it phrases the sub-question.

Failure is typed, not collapsed

REASON	RETRYABLE	WHAT THE AGENT DOES
not_available	false	Relays the refusal and stops. Restricted or absent data — rewording cannot help
rejected_by_guard	true	Rephrases more precisely and tries once more
error	false	Reports the request could not be completed. Only the exception type crosses the boundary

Why each agent has its own guard



The leads domain is deliberately narrower. Conversion is recorded on the lead itself, so funnel questions need no access to deals — which makes the boundary real rather than decorative. A question genuinely spanning both is a routing problem, not a reason for an agent to widen its own surface.

The layers

COMPONENT	RESPONSIBILITY	FILE
Supervisor	Classifies the turn into a domain, or answers out-of-scope directly	graph/supervisor.py
Domain	Binds tables, guard scope, rules and prompt into one declaration	graph/agents/domain.py
Domain agent	Picks tools, reads results, writes the answer. Never writes SQL	graph/agents/deals.py
sql_query	The only path to CRM data; caps the payload and types failures	tools/sql.py
Analysis tools	Arithmetic on numbers already held — no database access	tools/analysis.py
SQL agent	Question → one SELECT, or a typed refusal	sql/agent.py
Catalogue	Schema, business rules, relationships — and the guard's allowlist	sql/catalogue.py
SQLGuard	Validates every query before it reaches PostgreSQL	sql/guard.py
Repository	The single execution path; always validates first	db/repositories/sql.py

What the guard does not check: columns.

Masked money columns are held back because they are absent from the catalogue — the model never sees them as queryable — and the prompt instructs a refusal when one is requested. Both hold across the eval suite. But the guard validates tables and functions, not projections, so `SELECT *` is not blocked. Against the real `mytai` schema, where those columns physically exist, that is the gap. A known and accepted position, not an oversight.